15.1 函數、表示式與電路是三個層次

切換函數（switching function）是 $f:\{0,1\}^n\to\{0,1\}$。輸入依序寫成 $x_1,\dots,x_n$，共有 2^n 種 assignment（賦值），完整真值表指定每一列的輸出。每列有 2 種選擇，所以不同切換函數共有 2^{2^n} 個。

Boolean expression（布林表示式）用 AND、OR、NOT 表示函數。本章以 $xy=x\wedge y$、$x+y=x\vee y$、$\bar{x}=\neg x$ 記號；這裡 $+$ 不是整數加法，也不是 XOR。Literal（文字項）是變數或其補數；product term（乘積項）是 literals 的 AND；SOP（sum of products，積之和）是乘積項的 OR。

不同字串可能表示同一函數，例如 $AB+A\bar{B}=A$；不同電路也可能實現同一函數。先證明函數等價，再依 gate 數、fan-in（輸入腳數）、邏輯深度等目標設計電路。這些成本不自動等同於表示式的 literal 數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.2 Minterm：精確辨認一列

固定 bit 次序 A,B,C，index 為 $4A+2B+C$。對 assignment $a=(a_1,\dots,a_n)$，定義

$$\ell_i(a)=\begin{cases}x_i, & a_i=1,\\ \bar{x}_i, & a_i=0,\end{cases}\qquad m_a=\bigwedge_{i=1}^{n}\ell_i(a).$$

這不使用「$x^0=\bar{x}$」的特殊冪次約定。

若輸入等於 a，每個 literal 都是 1，因此 $m_a=1$；若某一 bit 不同，對應 literal 為 0，因此 $m_a=0$。故 m_a 恰在第 a 列為真，不同 minterms 互斥。

例如 $a=101$ 的 minterm 是 $A\bar{B}C=m_5$，而 011 是 $\bar{A}BC=m_3$。三個變數的每個 minterm 都含三個 literals；漏掉一個變數就會同時接受兩列，已不是 minterm。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.3 Canonical DNF 的存在與唯一性

對完整指定的函數，

$$f = \bigvee_{a : f(a)=1} m_a.$$

在任意輸入 b，所有 $a \neq b$ 的項都為 0；若 $f(b)=1$，右式含 $m_b=1$，否則不含。這逐列證明等式。

唯一性指「每列都展開成完整 minterm，忽略排列與重複」後的 canonical DNF：在輸入 b 唯一可能為真的項是 m_b，所以它是否出現由 $f(b)$ 唯一決定。一般 DNF 並不唯一，也不一定最小。

例 $f = \Sigma m(3,5,7)$：

ABC	000	001	010	011	100	101	110	111
f	0	0	0	1	0	1	0	1

因此 $f = \bar{A}BC + A\bar{B}C + ABC$，化簡則為 $BC + AC$。本章採 empty OR $=0$、empty AND $=1$，包含常數函數與 $n=0$ 的邊界。

15.4 Maxterm 與 canonical CNF

對 assignment a，改用在該列為 0 的 literals：

$$q_i(a) = \begin{cases} \bar{x}_i, & a_i = 1, \\ x_i, & a_i = 0, \end{cases} \qquad M_a = \bigvee_i q_i(a).$$

在 a 所有 literals 都為 0；在其他列至少有一個 literal 為 1。所以 M_a 只在 a 為 0。

$$f = \bigwedge_{a : f(a)=0} M_a$$

的證明同樣逐列：輸入 b 若為 false row，含因子 $M_b=0$；若為 true row，每個因子都是 1。這是 canonical CNF（合取標準型）。

例如 XOR 的 false rows 是 00、11，故

$$A \oplus B = (A+B)(\bar{A}+\bar{B}) = A\bar{B} + \bar{A}B.$$

DNF 列 true rows，CNF 列 false rows；不同函數兩種標準型的長度可能差很多。

15.5 抽象 Boolean algebra 的完整約定

本章的 Boolean algebra 是有界、分配且每個元素都有補元的 lattice。具體地，非空集合 B 上有二元運算 $\vee, \wedge$、常數 $0, 1$，對所有 $x, y, z \in B$ 滿足：

性質	join 形式	meet 形式
交換律	$x \vee y = y \vee x$	$x \wedge y = y \wedge x$
結合律	$(x \vee y) \vee z = x \vee (y \vee z)$	$(x \wedge y) \wedge z = x \wedge (y \wedge z)$
吸收律	$x \vee (x \wedge y) = x$	$x \wedge (x \vee y) = x$
界與恆等律	$x \vee 0 = x,\ x \vee 1 = 1$	$x \wedge 1 = x,\ x \wedge 0 = 0$
分配律	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

每個 x 至少有一個補元 y，滿足 $x \vee y = 1,\ x \wedge y = 0$。後面證明補元唯一，記作 $\bar{x}$。允許只有一個元素的退化代數 $0 = 1$；若要求非零代數會明寫 $0 \neq 1$。

15.6 由代數運算得到偏序

定義 $x \leq y \iff x \wedge y = x$。由吸收律令 $y = 0$ 得 $x \wedge x = x$，其對偶給 $x \vee x = x$，所以有反身性。

若 $x \leq y$ 且 $y \leq x$，則 $x = x \wedge y = y \wedge x = y$，故反對稱。若 $x \leq y \leq z$，

$$x \wedge z = (x \wedge y) \wedge z = x \wedge (y \wedge z) = x \wedge y = x,$$

故傳遞。另由吸收律與代入，

$$x \leq y \iff x \vee y = y.$$

向右：$x \vee y = (x \wedge y) \vee y = y$。向左：$x \wedge y = x \wedge (x \vee y) = x$。

$x \wedge y$ 是共同下界；若 $z \leq x, y$，則 $z \wedge (x \wedge y) = (z \wedge x) \wedge y = z$，故為最大下界。對偶可得 $x \vee y$ 為最小上界。因此 finite join（有限個 join）有明確的最小上界意義。

15.7 補元唯一、De Morgan 與 duality

設 b, c 都是 a 的補元。分配律給

$$b = b \wedge 1 = b \wedge (a \vee c) = (b \wedge a) \vee (b \wedge c) = b \wedge c.$$

同理 $c = b \wedge c$，故 $b = c$。因為 a 也是 $\bar{a}$ 的補元，唯一性給 $\bar{\bar{a}} = a$；也有 $\bar{0} = 1, \bar{1} = 0$。

令 $d = \bar{x} \vee \bar{y}$。逐項分配得

$$(x \wedge y) \wedge d = 0, \qquad (x \wedge y) \vee d = (x \vee d) \wedge (y \vee d) = 1.$$

所以由補元唯一性，$\overline{x \wedge y} = \bar{x} \vee \bar{y}$。對偶得 $\overline{x \vee y} = \bar{x} \wedge \bar{y}$。

Duality（對偶原理）交換 $\wedge/\vee$ 及 $0/1$。公理表在交換後仍是同一組公理，所以一個從公理推出的恆等式，整個推導交換後仍有效。這不是對函數逐點取補數。

15.8 化簡必須保留推理與成本

例：

$$\begin{aligned}
(A+B)(A+\bar{B})(\bar{A}+B) &= (A+B\bar{B})(\bar{A}+B) && \text{分配律} \\
&= A(\bar{A}+B) && \text{互補律、恆等律} \\
&= A\bar{A}+AB && \text{分配律} \\
&= AB. && \text{互補律、恆等律}
\end{aligned}$$

每行是所有輸入上都相等，不是只驗某幾列。

若 $E = (A+B)C$，則 dual 是 $(AB)+C$，complement 是 $\bar{E} = (\bar{A}\bar{B})+\bar{C}$。例如輸入 $A = B = C = 1$，dual 為 1，complement 為 0，直接排除兩者相同。

這些定律也適用於抽象 Boolean algebra；但「把所有 2^n 列列完」只直接驗證二元素切換函數。抽象結構的定理仍須由公理或有效的表示定理推導。

15.9 Implicant、prime 與「最小 SOP」

先討論完整指定的函數。排除同時含 $x, \bar{x}$ 的恒零項，乘積項 P 若滿足 $P = 1 \Rightarrow f = 1$，稱 implicant（蘊含項）。刪除 literal 會擴大接受的 assignments；若任何 literal 都不能再刪而仍合法，稱 prime implicant（質蘊含項）。

例如 $f = \Sigma m(3, 5, 7)$，ABC 是 implicant，但不是 prime：刪去 A 得 BC，刪去 B 得 AC，都不碰 false row；而 AC 再刪任一 literal 都會碰 false row。

本章若未另說，「最小 SOP」採先最少乘積項、再最少 literal 數的字典序成本；若題目只問 literal 最少，則依題意。任意合法項都能在有限步內擴大成某個 prime，且擴大不增加項數或 literal 數。因此這兩種成本都存在只用 primes 的最佳解。

這不保證最佳解唯一，也不保證逐次選最大 group 能得到全域最佳解。

15.10 Prime chart 與必要的覆蓋

對 $f = \Sigma m(3, 5, 7)$，合法的最大 cubes 只有 BC 和 AC：

prime	$m_3 = 011$	$m_5 = 101$	$m_7 = 111$	literals
BC	✓		✓	2
AC		✓	✓	2

m_3 只能由 BC 覆蓋，m_5 只能由 AC 覆蓋，所以兩者都是 essential prime implicants（必要質蘊含項）。每個 SOP 解都可擴大為 prime cover，故至少需要這兩項。

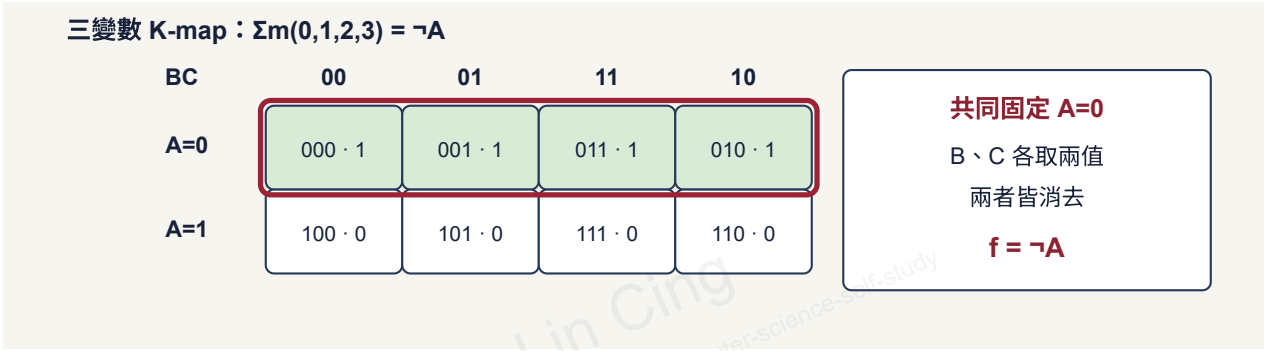
$$f = AC + BC$$

達成 2 項、4 literals，這才是最小 SOP 的證明。

提因式得 $(A + B)C$，在一般表示式中只有 3 次 literal 出現，但它不是 SOP，不能拿來宣稱「最小 SOP 只有 3 literals」。表示式家族與成本必須一起固定。

15.11 K-map：Gray code 與四格消元

三變數 K-map 取 rows $A = 0, 1$、columns $BC = 00, 01, 11, 10$。相鄰欄只差一 bit，最左、最右欄也相鄰。對 $f = \Sigma m(0, 1, 2, 3)$，四格在 $A = 0$ 的同一 row，B, C 自由變動，故 $f = \bar{A}$。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四格圈選的代數核對

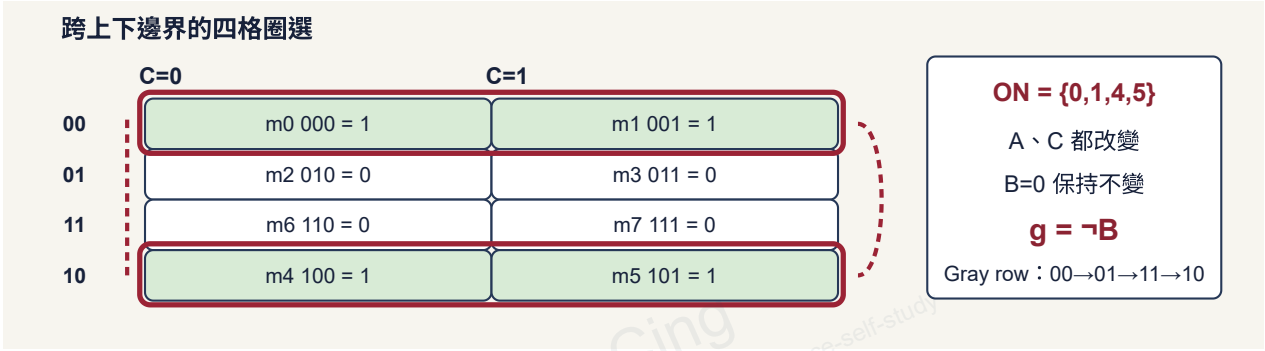
$$\bar{A}(\bar{B}\bar{C} + \bar{B}C + B\bar{C} + BC) = \bar{A}(\bar{B} + B)(\bar{C} + C) = \bar{A}.$$

圖中的圈選與這次分配完全對應，沒有把不同 row 誤認為同一 group。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.12 換一個軸配置，看跨邊界鄰接

取 rows $AB = 00, 01, 11, 10$，columns $C = 0, 1$，並令 $g = \Sigma m(0, 1, 4, 5)$。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼上下邊界可以圈在一起？

m_0, m_1 在最上 row，m_4, m_5 在最下 row；$AB = 00$ 與 10 只差 A，所以兩端相鄰。四格中 A, C 可自由變化，$B = 0$ 固定，故 $g = \bar{B}$。跨邊界呈現的是二進位 cube 既有的鄰接，不是任意連接。

合法 group 固定某些 bit，其餘 bit 全部自由，形成大小 2^k 的 cube，消去 k 個 literals。三變數圖可用含 $1, 2, 4, 8$ 格的矩形，允許 wrap-around 與重疊；只有格數是 2 的冪仍不夠，L 形等不完整 cube 不合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.13 沒有 essential prime，也能證明最小

令 $h = \Sigma m(1,2,3,4,5,6)$，只有 000 與 111 為 0。六個 primes 為：

prime	覆蓋 ON indices	prime	覆蓋 ON indices
$\bar{A}B$	2,3	$A\bar{B}$	4,5
$\bar{A}C$	1,3	$A\bar{C}$	4,6
$B\bar{C}$	2,6	$\bar{B}C$	1,5

每個 ON 列恰被兩個 primes 覆蓋，所以沒有 essential prime。每個單 literal 都包含 000 或 111，故不合法；任何合法項至多覆蓋兩個 ON 列，因此至少要 3 項，每項至少 2 literals。

下列兩個 covers 都達到 3 項、6 literals：

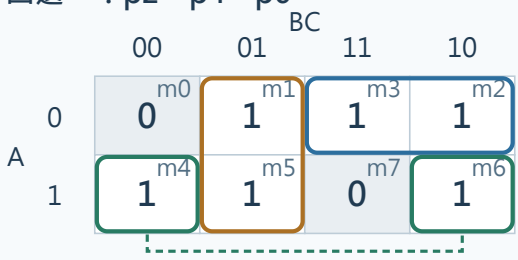
$$h = \bar{A}B + A\bar{C} + \bar{B}C = \bar{A}C + A\bar{B} + B\bar{C}.$$

這是兩個不同最佳 SOP。它也說明「找完 essential primes」未必已經完成最小化，剩餘 chart 仍是 cover 選擇問題。

同一張 K-map 的兩組最佳圈選

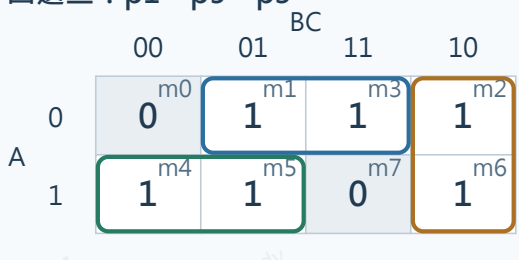
Rows 為 A=0,1，columns 依 Gray code 排成 BC=00,01,11,10；每種顏色代表一個乘積項。

圈選一：p2、p4、p6



p2: (¬A)B {2,3}
p4: A(¬C) {4,6}
p6: (¬B)C {1,5}

圈選二：p1、p5、p3



p1: (¬A)C {1,3}
p5: A(¬B) {4,5}
p3: B(¬C) {2,6}

比較兩組最佳圈選

左圖兩端的綠格是一組跨邊界相鄰的 pair。兩圖都覆蓋六個 1 格、避開兩個 0 格，各用三項、每項兩個 literals，達到前頁下界。第 7 題詳解再證明：最少項數的 prime covers 只有這兩組。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.14 從 K-map 到一般 cube 演算法

以 0、1、– 表示 cube：例如 $1{-}1$ 是 AC。兩個 cubes 只有在 dash 的位置相同、其餘固定位元恰差一 bit 時才可合併，例如 $101, 111 \to 1{-}1$。合併後是兩者的聯集；不是把不同 dash 配置任意湊在一起。

從所有合法 minterms 開始分層合併；記錄曾被更大 cube 包含的項，不能再擴大的合法 cube 就是 prime。完整性可由自由 bit 數歸納：任一 k-自由 bit 的合法 cube，固定其中一個自由 bit 的 0/1 子 cube 也合法；兩半都能在前一層產生，再合併回原 cube。

接著建立 prime chart：columns 是必須覆蓋的 ON 列，rows 是 primes。選 row 覆蓋全部 columns，以項數／literal 成本最佳化；可用窮舉、分支界限或精確求解 set cover；允許不同項重疊覆蓋同一列，並非要求每列恰好覆蓋一次的 exact cover。僅按每個 row 覆蓋格數排序，是 heuristic，沒有一般最優性保證。

變數多時，真值表、primes 及 cover 搜尋都可能很大；小型 K-map 的方便不代表任意規模都有簡單多項式最小化法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.15 Don't-care：規格先於化簡

令 ON set F、OFF set R、don't-care set D 兩兩不交，且聯集是全部 assignments。合法實作 g 必須在 F 為 1、在 R 為 0；在 D 可選 0 或 1，稱 completion（完整化）。

部分規格的合法 cube 只須不碰 R。可用 $F \cup D$ 產生 primes，再只要求覆蓋 F；完全只覆蓋 D 的 prime 不必選。這與完整指定函數的條件 $P \Rightarrow f$ 不同，因為原規格在 D 尚無 f 值。

例如 $F = \{0, 2, 4\}, D = \{1, 5\}, R = \{3, 6, 7\}$：

$$g = \bar{B} + \bar{A}\bar{C}.$$

$\bar{B}$ 接受 0,1,4,5，$\bar{A}\bar{C}$ 接受 0,2；聯集是 0,1,2,4,5，確實不含任何 OFF 列，並將兩個 don't-care 列選成 1。

若之後系統規格開始要求輸入 1 或 5 的特定輸出，這已是新規格，必須檢查所選 completion 是否符合。

15.16 Don't-care 數例的最小性

同一規格 $F = \{0, 2, 4\}, D = \{1, 5\}, R = \{3, 6, 7\}$，成本改問「總 literal 數最少」。

六個單 literal 中，只有 $\bar{B}$ 不碰 OFF set；它漏掉 ON 列 $2 = 010$。其餘 $A, \bar{A}, B, C, \bar{C}$ 分別碰到 OFF 列 6、3、3、3、6，均不能單獨作合法項。

若 SOP 只有至多兩個 literal 出現：

- 兩個一 literal 項都只能是 $\bar{B}$，仍漏掉 2。
- 一個二 literal 項無法同時覆蓋 0,2,4，因為三列共同固定的 bit 只有 $C = 0$，不能再固定第二個 bit；即使放寬成 $\bar{C}$，又會碰到 6。
- 常數 1 碰到所有 OFF 列，常數 0 漏掉 ON 列。

所以至少需 3 literals；$\bar{B} + \bar{A}\bar{C}$ 恰達下界。合法與最小是兩項不同責任，逐列驗真只能證明前者。

15.17 Shannon expansion 與通用 gates

固定第一個變數 x，令 $f_0 = f|_{x=0}, f_1 = f|_{x=1}$。則

$$f = \bar{x} f_0 + x f_1.$$

在 $x = 0$ 右式就是 f_0，在 $x = 1$ 就是 f_1，因此等式對全部輸入成立。這也是 2-to-1 multiplexer（多工器）的選擇語義。

NAND 定義 $x \uparrow y = \overline{xy}$。只用二輸入 NAND：

$$\bar{x} = x \uparrow x, \quad xy = (x \uparrow y) \uparrow (x \uparrow y), \quad x + y = (x \uparrow x) \uparrow (y \uparrow y).$$

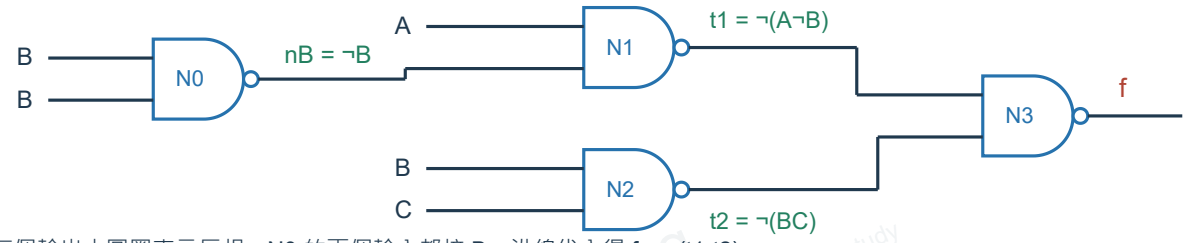
既然 AND、OR、NOT 可由 NAND 實現，canonical DNF 又能表任意切換函數，所以對 $n \geq 1$ 的切換函數，NAND 是 functionally complete（功能完備）的 gate 種類。由對偶論證 NOR 也完備。$n = 0$ 時若電路沒有任何輸入及常數來源，無法只用二輸入 NAND 接出常數；須另外允許常數來源。當 $n \geq 1$ 時，可以由 $x \uparrow (x \uparrow x) = 1$ 和 $1 \uparrow 1 = 0$ 產生兩個常數。

此為「存在一種電路」的證明；不是最少 gate 數的證明，也沒有計入實際負載與傳播延遲。

15.18 真正接線：四個 NAND 實現同一函數

對 $f = A\bar{B} + BC$，四個 NAND 的輸入腳與中間訊號如下。

同一函數 f = A¬B + BC：四個二輸入 NAND 的實際接線



每個輸出小圓圈表示反相；N0 的兩個輸入都接 B。沿線代入得 f = ¬(t1·t2)。

沿接線寫出每個中間訊號

$$n_B = \mathrm{NAND}(B, B) = \bar{B}, \quad t_1 = \mathrm{NAND}(A, n_B) = \overline{A\bar{B}}, \quad t_2 = \mathrm{NAND}(B, C) = \overline{BC}.$$

最後 $f = \mathrm{NAND}(t_1, t_2) = \overline{\overline{A\bar{B}}\,\overline{BC}}$，由 De Morgan 得所需 OR。

N0 的兩個 input pins 都接 B，所以充當 inverter；若外部已提供 $\bar{B}$，可省 N0。本圖有 4 個二輸入 NAND，最大邏輯深度為從 B 經 N0、N1、N3 的 3 層。深度只計 gate 層數，實際時間還要各 gate 的延遲模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.19 接線驗證必須包含中間訊號

沿上一頁每條 wire 計算，不能遺漏 gate 的輸出 bubble。先檢查 $A = 0$ 的四列：

ABC	n_B	t_1	t_2	NAND 輸出	$A\bar{B} + BC$
000	1	1	1	0	0
001	1	1	1	0	0
010	0	1	1	0	0
011	0	1	0	1	1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

其餘輸入與延遲限制

再檢查 $A = 1$ 的四列，兩張表的最後兩欄逐列相同，故驗證整個函數。

ABC	n_B	t_1	t_2	NAND 輸出	$A\bar{B} + BC$
100	1	0	1	1	1
101	1	0	1	1	1
110	0	1	1	0	0
111	0	1	0	1	1

四個相同 NAND gates 不代表四個相同延遲串聯，因 N1、N2 有平行分支；fan-out、wire delay 與 hazard（輸出暫態）也未由此靜態真值表保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.20 抽象例子：集合與整除

$\mathcal{P}(S)$ 以 union 為 join、intersection 為 meet、相對 S 的 complement 為補元；零元是 $\varnothing$，單位元是 S。這直接滿足公理。

另一個例子是 30 的正因數集合

$$B = \{1, 2, 3, 5, 6, 10, 15, 30\}, \quad x \wedge y = \gcd(x, y), \quad x \vee y = \operatorname{lcm}(x, y).$$

把每個因數對應到它包含的質因數子集合：$6 \leftrightarrow \{2, 3\}$、$10 \leftrightarrow \{2, 5\}$。因為 $30 = 2 \cdot 3 \cdot 5$ 沒有重複質因數，gcd/lcm 對應 intersection/union，補元是 $30/x$。

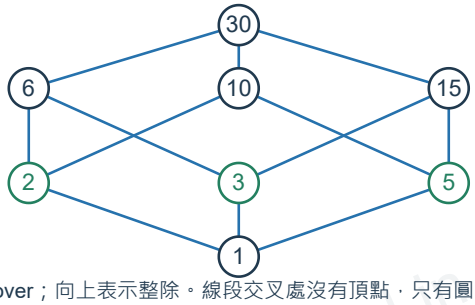
這裡的零元是數字 1，單位元是數字 30，不可與整數運算的 0、1 混用。例如 $6 \wedge 10 = 2,\ 6 \vee 10 = 30,\ \bar{6} = 5$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.21 Hasse 圖與 atoms

Atom（原子）是非零元素中的極小元素：$a \neq 0$，且 $0 < y \leq a \Rightarrow y = a$。30 的因數例中，atoms 是互不可比的 2、3、5。

30 的正因數：整除偏序就是三個 atoms 的子集合偏序



零元 1；單位元 30（依整除序）

atoms：2 ↔ {2}，3 ↔ {3}，5 ↔ {5}

meet = gcd；join = lcm

6 ∧ 10 = 2；6 ∨ 10 = 30

¬6 = 5；{2,3} 的補集為 {5}

每線皆為 cover；向上表示整除。線段交叉處沒有頂點，只有圓圈才是元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由 Hasse 圖辨認 atoms 與補元

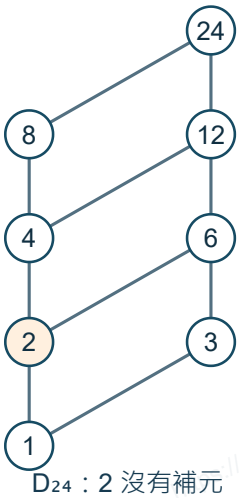
Atom 不必小於其他所有非零元素，不能稱為 minimum。沿 cover 向上是加入一個尚未出現的質因數；並非所有可整除的兩點都直接連邊。三個 atoms 的 8 個子集合剛好給 8 個因數。

並非每個因數 lattice 都是 Boolean：12 的因數中，2 若有補元 b，必有 $\gcd(2,b)=1, \operatorname{lcm}(2,b)=12$。可與 2 互質的因數只有 1、3，其 lcm 分別是 2、6，都不是 12，所以 2 沒有補元。

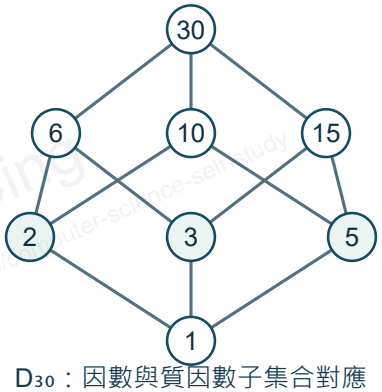
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相同的元素數，不一定是布林代數

令 D_n 為 n 的正因數，以整除為偏序，meet 是 gcd、join 是 lcm；底元為 1，頂元為 n。比較 D_{24} 與 D_{30}：兩者都恰有 8 個元素，但覆蓋關係不同。



D_{24}：2 沒有補元



D_{30}：因數與質因數子集合對應

圖中線段只連接覆蓋關係；向上代表整除，不將所有可傳遞的整除關係另畫成邊。

為何 D_{24} 不是布林代數？

若 2 的補元是 d，則必須同時滿足 $\gcd(2, d) = 1$ 和 $\operatorname{lcm}(2, d) = 24$。在 D_{24} 中與 2 互質的因數只有 1 和 3；其 lcm 分別為 2 和 6，都不是 24。因此 2 沒有補元，D_{24} 不是布林代數。

另一方面，$D_{24} = \{1, 2, 3, 4, 6, 8, 12, 24\}$ 有 8 個元素。**元素個數是 2 的冪，並不足以保證一個格是布林代數。**

為何 D_{30} 是布林代數？

因 $30 = 2 \cdot 3 \cdot 5$ 是平方自由數，每個正因數唯一對應集合 $\{2, 3, 5\}$ 的一個子集：把因數分解成所含的質因數即可。例如 $6 \leftrightarrow \{2, 3\}$。

這個對應是雙射；gcd 恰對應交集，lcm 恰對應聯集，而 $30/d$ 對應集合補集。因此它是到 $\mathcal{P}(\{2, 3, 5\})$ 的布林代數同構。三個 atoms 是 2、3、5；共有 $2^3 = 8$ 個元素。反向不能只由元素個數推出同構，D_{24} 就是反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.22 有限代數：每個非零元素下方有 atom

令 B 是有限 Boolean algebra，且 $x \neq 0$。集合 $\{y : 0 < y \leq x\}$ 非空，因為包含 x；有限偏序必有極小元素 a。若 $0 < z \leq a$，傳遞性給 $z \leq x$，極小性迫使 $z = a$，故 a 是 atom。

若 a, b 為不同 atoms，則 $a \wedge b = 0$。否則 $0 < a \wedge b \leq a, b$，由 atom 定義同時得到 $a \wedge b = a = b$，矛盾。

注意「不同 atoms meet 為 0」不代表 lattice 中每個非零元素都是 atom。例如 30 的因數中，6 是兩個 atoms 的 join，不是 atom。有限性也不是裝飾：若結構無限，下降鏈可能沒有極小非零元素，以上取法就失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.23 每個元素恰是它下方 atoms 的 join

令 $A_x = \{a : a \text{ 是 atom 且 } a \le x\}$，設 $s = \bigvee A_x$，空 join 定為 0。因每個 $a \le x$，有 $s \le x$。

若 $s \neq x$，則 $x \wedge \bar{s} \neq 0$：否則由

$$x = x \wedge (s \vee \bar{s}) = (x \wedge s) \vee (x \wedge \bar{s}) = s$$

矛盾。由前頁，$x \wedge \bar{s}$ 下方存在 atom a。它滿足 $a \le x$，故 $a \in A_x$ 且 $a \le s$；同時 $a \le \bar{s}$，所以 $a \le s \wedge \bar{s} = 0$，又矛盾。

因此

$$x = \bigvee_{a \le x,\ a \text{ atom}} a.$$

特別令 $x = 1$，得到全部 atoms 的 join 是 1。這是有限表示定理的核心，不能只由幾張 Hasse 圖歸納宣告。

15.24 為什麼 atom 對 join 表現得像單一元素

對 atom a，由分配律

$$a \wedge (x \vee y) = (a \wedge x) \vee (a \wedge y).$$

每個 $a \wedge x$ 不是 0 就是 a。因此

$$a \le x \vee y \iff a \le x \text{ 或 } a \le y.$$

其中反向由上界性；正向若兩個 meets 都是 0，右式會是 0，與 $a \neq 0$ 矛盾。

同理 $a \le x \wedge y \iff a \le x$ 且 $a \le y$。對補元，$a = (a \wedge x) \vee (a \wedge \bar{x})$，至少一項是 a，兩項又不能同時是 a，所以

$$a \le \bar{x} \iff a \not\le x.$$

這裡使用 atom 與分配律；不能把「$z \le x \vee y$ 必有 $z \le x$ 或 $z \le y$」誤當任意元素都成立。

15.25 有限 Boolean algebra 的表示定理

令 A 為全部 atoms，定義 $\Phi : B \to \mathcal{P}(A)$， $\Phi(x) = \{a \in A : a \le x\}$。前頁給

$$\Phi(x \wedge y) = \Phi(x) \cap \Phi(y), \quad \Phi(x \vee y) = \Phi(x) \cup \Phi(y), \quad \Phi(\bar{x}) = A \setminus \Phi(x).$$

又有 $\Phi(0) = \varnothing, \Phi(1) = A$，故它保留所有運算。

若 $\Phi(x) = \Phi(y)$，前兩頁的 atom-join 定理給 $x = y$，所以 injective。對任意 $T \subseteq A$，令 $x = \bigvee T$。其中每個 atom 都在 $\Phi(x)$；若另一 atom $b \le x$，反覆使用前頁 join 性質得到 $b \le a$ 對某個 $a \in T$，故 $b = a$。空 T 時 $x = 0$ 也成立。因此 $\Phi(x) = T$，所以 surjective。

結論：$B \cong \mathcal{P}(A)$。若有 k 個 atoms，則 $|B| = 2^k$；運算必須透過這個同構對照，不只是元素數目相同。退化代數有 $k = 0$，對應 $\mathcal{P}(\varnothing) = \{\varnothing\}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.26 不是每個無限 Boolean algebra 都由 atoms 組成

有限表示定理用了「非零元素下方必有 atom」。無限代數可有反例：取所有長度 1 的圓周上，端點在二進分數的半開弧段所成的有限聯集，加上空集；補集、交、聯仍在其中，所以形成 Boolean algebra。

任一非空元素含某個長度為 2^{-r} 的半開小弧段；把它二等分後，得到嚴格較小且非空的元素。因此沒有 atom。但它含空集與整個圓周，並非只有一元素的代數。

所以不能不加有限前提，就說「沒有 atoms 必為零代數」或「任何 Boolean algebra 都是其 atoms 的全部子集合」。無限 Boolean algebra 的一般表示定理需要不同對象；本章不以有限證明冒充該一般結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.27 Boolean algebra 與 Boolean ring

在集合例中，對稱差對應 XOR。一般 Boolean algebra 可定義

$$x \oplus y = (x \wedge \bar{y}) \vee (\bar{x} \wedge y), \qquad xy = x \wedge y.$$

在有限情況，經 Φ 對應到集合的對稱差與交集，立即得到交換 ring，零元為 Boolean 的 0，單位元為 Boolean 的 1，且 $x^2 = x$。

反方向，若有含單位元的 ring 且每個元素滿足 $x^2 = x$，則 $(x+x)^2 = x + x$ 給 $4x = 2x$，故 $2x = 0$。由 $(x+y)^2 = x + y$ 得 $xy + yx = 0$，再由每個元素滿足 $2z = 0$ 推出 $xy = yx$，此論證也包含退化零環。可定義 $x \wedge y = xy,\ \bar{x} = 1 + x,\ x \vee y = x + y + xy$。

本頁連結第 14 章，但必須分清兩種加號：ring 的 $+$ 是 XOR，Boolean algebra 的 $\vee$ 是 OR；在非退化代數 $0 \neq 1$ 中，兩者於 $x = y = 1$ 的結果分別是 0、1；退化代數的兩個常數相同。

15.28 Boolean ring 反方向的結構驗證

接續上一頁，已知 ring 交換、$2x = 0$、$x^2 = x$。定義 $x \leq y \iff xy = x$。反身由冪等，反對稱由交換；若 $xy = x, yz = y$，則 $xz = xyz = xy = x$，故為偏序。又 $0 \leq x \leq 1$。

xy 是最大下界：$(xy)x = xy$，若 $zx = z, zy = z$，則 $zxy = z$。令 $j = x + y + xy$，則 $xj = x,\ yj = y$，所以是共同上界；若 $xz = x, yz = y$，則 $jz = x + y + xy = j$，故是最小上界。偏序中的最大下界、最小上界唯一，自動給交換、結合、吸收律。

補元 $1 + x$ 滿足 $x(1+x) = 0$，且 $x \vee (1+x) = 1$。兩個分配律可直接算：

$$x(y \vee z) = xy + xz + xyz = (xy) \vee (xz),$$
$$(x \vee y)(x \vee z) = x + yz + xyz = x \vee (yz).$$

展開時使用 $x^2 = x$、$2t = 0$。因此確實滿足前面完整 Boolean algebra 公理，而不只是一組看似相似的運算公式。

15.29 自學檢查：可以重建什麼

本章需要能由規格重建結果：

- 指定 bit 次序，從每個 true/false row 寫出 minterm/maxterm，證明標準型逐列等價。
- 分清合法 implicant、prime、essential prime 與最佳 cover；在固定表示式家族和成本下給下界與達成解。
- 讀 K-map 的軸與 Gray code，解釋每個 group 固定了哪些 bit，並檢查沒有覆蓋 OFF 列。
- 沿 NAND 的實際 wire 推導中間訊號與全部輸出，不把靜態真值表誤當延遲或暫態保證。
- 從 Boolean 公理建立偏序、補元與 atoms，再完整證明有限表示，而非只背 2^k。

各項對應本章獨立練習及詳解；練習中的成本和有限性條件都是題目的一部分。

驗證化簡與表示

化簡後先逐列核對 ON 與 OFF；有不在乎項時，只在規格允許的輸入上選擇輸出。若宣稱最小，還需說明成本模型及下界。

參考資料與署名

Lin Cing，《離散數學》手寫筆記。 Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 15.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study